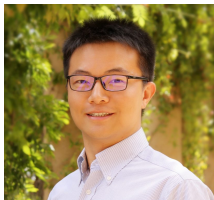


EfficientML.ai Lecture 06

Quantization

Part II



Song Han

Associate Professor, MIT
Distinguished Scientist, NVIDIA

 @SongHan_MIT



Lecture Plan

Today we will:

1. Review Linear Quantization.
2. Introduce **Post-Training Quantization (PTQ)** that quantizes a floating-point neural network model, including: channel quantization, group quantization, and range clipping.
3. Introduce **Quantization-Aware Training (QAT)** that emulates inference-time quantization during the training/fine-tuning and recover the accuracy.
4. Introduce **binary and ternary** quantization.
5. Introduce automatic **mixed-precision** quantization.

Neural Network Quantization

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	3:	2.00
1	1	0	3	2:	1.50
0	3	1	0	1:	0.00
3	1	2	2	0:	-1.00

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

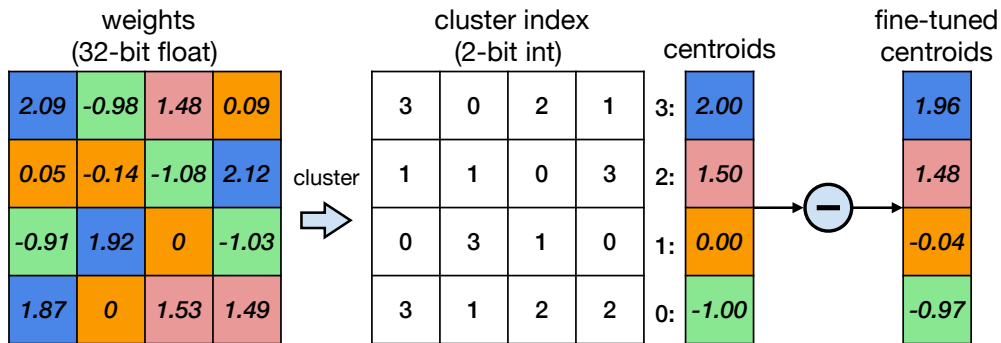
$(\dots) - (-1) \times 1.07$

**K-Means-based
Quantization**

**Linear
Quantization**

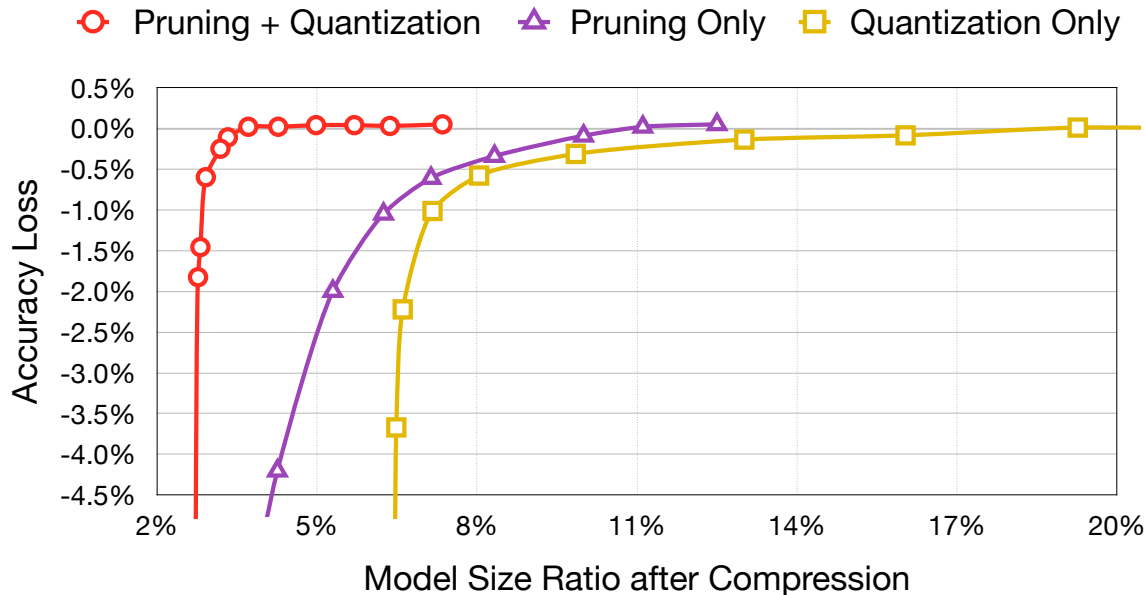
Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic

K-Means-based Weight Quantization



K-Means-based Weight Quantization

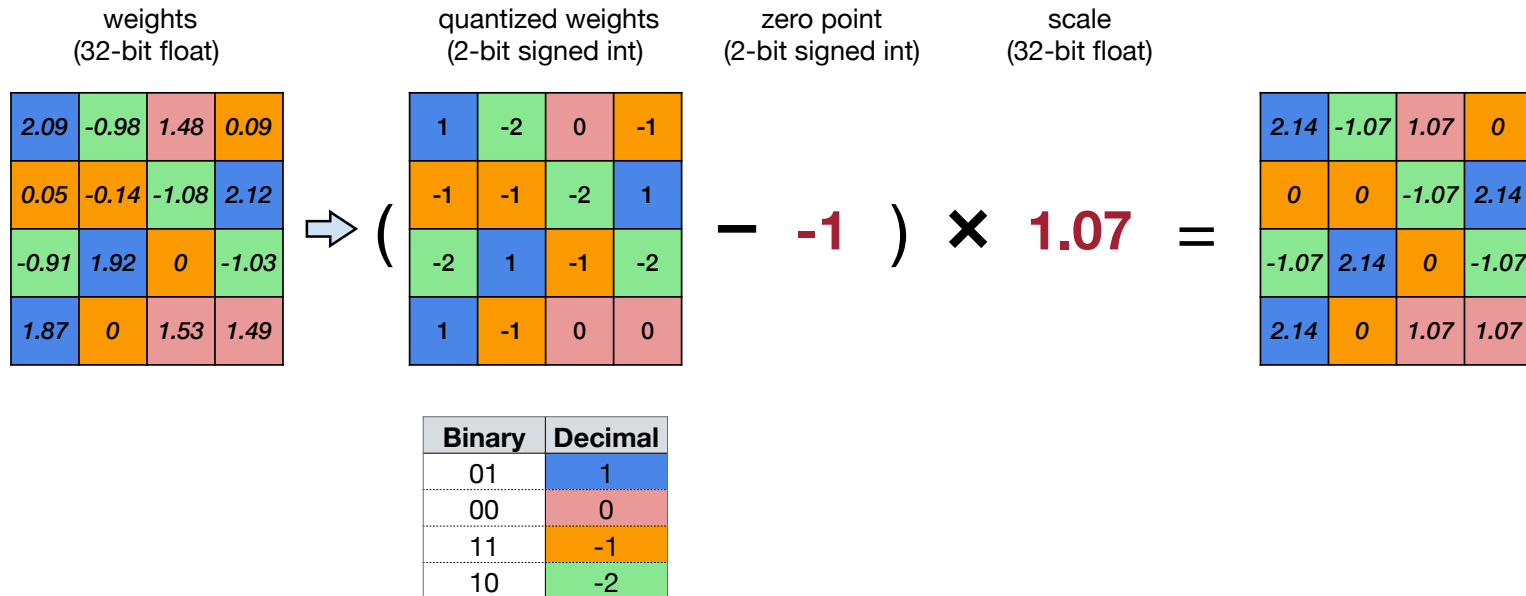
Accuracy vs. compression rate for AlexNet on ImageNet dataset



Deep Compression [Han et al., ICLR 2016]

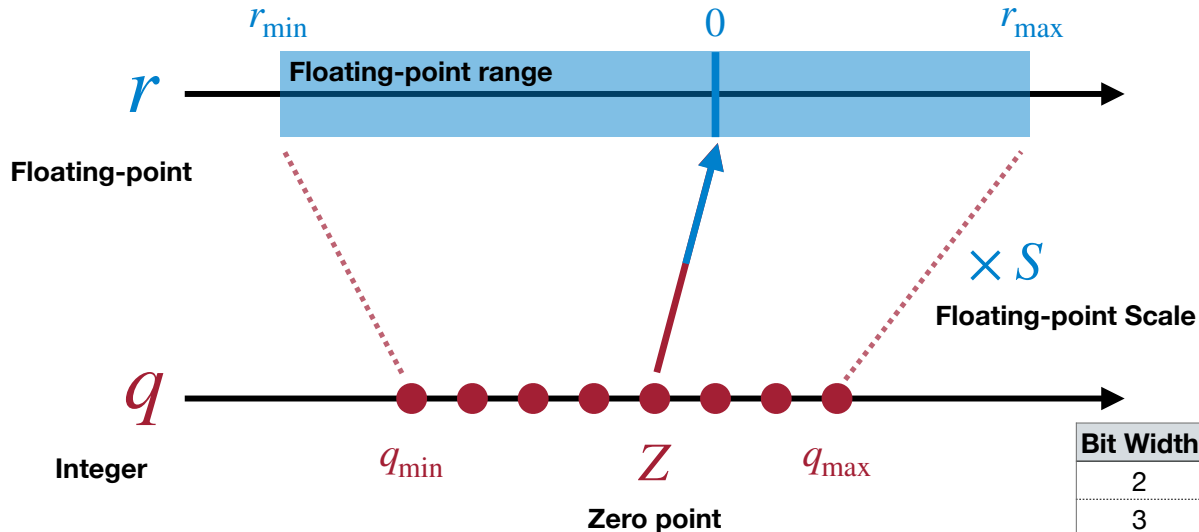
Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$



Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$



Bit Width	$q_{\min}$	$q_{\max}$
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

Linear Quantized Fully-Connected Layer

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- Consider the following fully-connected layer.

$$\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$$

$$Z_W = 0$$

$$Z_b = 0, \quad S_b = S_W S_X$$

$$\mathbf{q}_{bias} = \mathbf{q}_b - Z_X \mathbf{q}_W$$

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} \left(\mathbf{q}_W \mathbf{q}_X + \mathbf{q}_{bias} \right) + Z_Y$$

Rescale to N -bit Int N -bit Int Mult. N -bit Int
32-bit Int Add. Add

Note: both $\mathbf{q}_b$ and $\mathbf{q}_{bias}$ are 32 bits.

Linear Quantized Convolution Layer

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- Consider the following convolution layer.

$$Y = \text{Conv}(W, X) + b$$

$$\begin{aligned} Z_W &= 0 \\ Z_b &= 0, \quad S_b = S_W S_X \\ q_{bias} &= q_b - \text{Conv}(q_W, Z_X) \end{aligned}$$

$$q_Y = \frac{S_W S_X}{S_Y} \left(\text{Conv}(q_W, q_X) + q_{bias} \right) + Z_Y$$

Rescale to N -bit Int N -bit Int Mult. 32-bit Int Add. N -bit Int Add

Note: both q_b and q_{bias} are 32 bits.

Post-Training Quantization

How should we get the optimal linear quantization parameters (S, Z)?

Topic I: Quantization Granularity

Topic II: Dynamic Range Clipping

Topic III: Rounding

Post-Training Quantization

Topic I: Quantization Granularity

Topic II: Dynamic Range Clipping

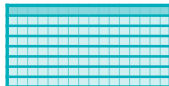
Topic III: Rounding

Quantization Granularity

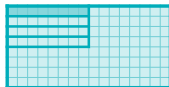
- Per-Tensor Quantization



- Per-Channel Quantization



- Group Quantization



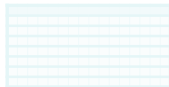
- Per-Vector Quantization
- Shared Micro-exponent (MX) data type

Quantization Granularity

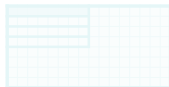
- **Per-Tensor Quantization**



- Per-Channel Quantization



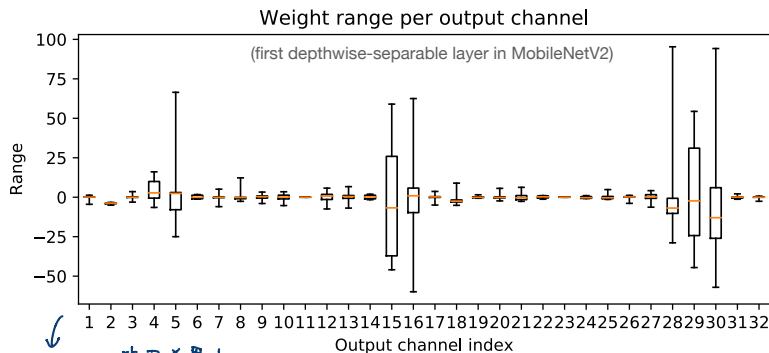
- Group Quantization



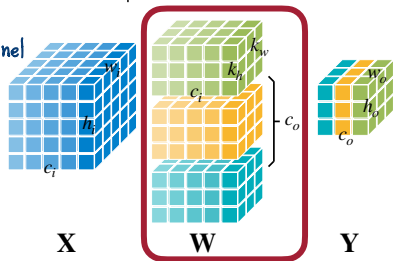
- Per-Vector Quantization

- Shared Micro-exponent (MX) data type

Symmetric Linear Quantization on Weights



output channel 範圍差異大
如果對這些差異大的 channel
用相同的 scaling factor,
無法讓所有 channel 都有
好的表現



- $|r|_{\max} = |\mathbf{W}|_{\max}$
- Using *single* scale S for whole weight tensor (**Per-Tensor Quantization**)
 - works well for large models
 - accuracy drops for small models
- Common failure results from
 - large differences (more than 100x) in ranges of weights for different output channels — outlier weight
- Solution: **Per-Channel Quantization**
為每個 channel 客製一個 scaling factor

Data-Free Quantization Through Weight Equalization and Bias Correction [Markus *et al.*, ICCV 2019]

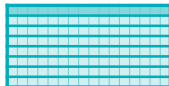
Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob *et al.*, CVPR 2018]

Quantization Granularity

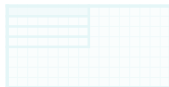
- Per-Tensor Quantization



- **Per-Channel Quantization**



- Group Quantization



- Per-Vector Quantization

- Shared Micro-exponent (MX) data type

Per-Channel Weight Quantization

Example: 2-bit linear quantization

	ic			
oc	2.09	-0.98	1.48	0.09
	0.05	-0.14	-1.08	2.12
	-0.91	1.92	0	-1.03
	1.87	0	1.53	1.49

Per-Channel Quantization

Per-Tensor Quantization

Per-Channel Weight Quantization

Example: 2-bit linear quantization

ic

	2.09	-0.98	1.48	0.09
OC	0.05	-0.14	-1.08	2.12
	-0.91	1.92	0	-1.03
	1.87	0	1.53	1.49

Per-Channel Quantization

Per-Tensor Quantization

$$|r|_{\max} = 2.12$$

$$S = \frac{|r|_{\max}}{q_{\max}} = \frac{2.12}{2^2 - 1} = 2.12$$

$\frac{2.09}{2.12}$ $\frac{-0.98}{2.12}$ $\downarrow$ 2-bit

1	0	1	0
0	0	-1	1
0	1	0	0
1	0	1	1

2.12	0	2.12	0
0	0	-2.12	2.12
0	2.12	0	0
2.12	0	2.12	2.12

Quantized

Reconstructed

$$\|W - Sq_W\|_F = 2.28$$

Per-Channel Weight Quantization

Example: 2-bit linear quantization

		<i>ic</i>				Per-Channel Quantization	
<i>oc</i>		2.09	-0.98	1.48	0.09	$ r _{\max} = 2.09$	$S_0 = 2.09$
		0.05	-0.14	-1.08	2.12	$ r _{\max} = 2.12$	$S_1 = 2.12$
		-0.91	1.92	0	-1.03	$ r _{\max} = 1.92$	$S_2 = 1.92$
		1.87	0	1.53	1.49	$ r _{\max} = 1.87$	$S_3 = 1.87$

Per-Tensor Quantization

$$|r|_{\max} = 2.12$$

$$S = \frac{|r|_{\max}}{q_{\max}} = \frac{2.12}{2^{2-1} - 1} = 2.12$$

1	0	1	0	2.12	0	2.12	0
0	0	-1	1	0	0	-2.12	2.12
0	1	0	0	0	2.12	0	0
1	0	1	1	2.12	0	2.12	2.12

Quantized

Reconstructed

$$\|W - Sq_W\|_F = 2.28$$

Per-Channel Weight Quantization

Example: 2-bit linear quantization

		<i>ic</i>				Per-Channel Quantization	
<i>oc</i>		2.09	-0.98	1.48	0.09	$ r _{\max} = 2.09$	$S_0 = 2.09$
		0.05	-0.14	-1.08	2.12	$ r _{\max} = 2.12$	$S_1 = 2.12$
		-0.91	1.92	0	-1.03	$ r _{\max} = 1.92$	$S_2 = 1.92$
		1.87	0	1.53	1.49	$ r _{\max} = 1.87$	$S_3 = 1.87$

1	0	1	0
0	0	-1	1
0	1	0	-1
1	0	1	1

Quantized

2.09	0	2.09	0
0	0	-2.12	2.12
0	1.92	0	-1.92
1.87	0	1.87	1.87

Reconstructed

$$\|\mathbf{W} - \mathbf{S} \odot \mathbf{q}_W\|_F = 2.08$$

Per-Tensor Quantization

$$|r|_{\max} = 2.12$$

$$S = \frac{|r|_{\max}}{q_{\max}} = \frac{2.12}{2^{2-1} - 1} = 2.12$$

1	0	1	0
0	0	-1	1
0	1	0	0
1	0	1	1

Quantized

2.12	0	2.12	0
0	0	-2.12	2.12
0	2.12	0	0
2.12	0	2.12	2.12

Reconstructed

$$\|\mathbf{W} - S\mathbf{q}_W\|_F = 2.28$$

Per-Channel Weight Quantization

Example: 2-bit linear quantization ↗ 缺: 需要更多空間, 因為有很多 scaling factor

		Per-Channel Quantization			
		ic			
oc		2.09	-0.98	1.48	0.09
		0.05	-0.14	-1.08	2.12
		-0.91	1.92	0	-1.03
		1.87	0	1.53	1.49
		$ r _{\max} = 2.09 \quad S_0 = 2.09$ $ r _{\max} = 2.12 \quad S_1 = 2.12$ $ r _{\max} = 1.92 \quad S_2 = 1.92$ $ r _{\max} = 1.87 \quad S_3 = 1.87$			

1	0	1	0
0	0	-1	1
0	1	0	-1
1	0	1	1

Quantized

2.09	0	2.09	0
0	0	-2.12	2.12
0	1.92	0	-1.92
1.87	0	1.87	1.87

Reconstructed

$$\|W - S \odot q_W\|_F = 2.08$$

Per-Tensor Quantization

$$|r|_{\max} = 2.12$$

$$S = \frac{|r|_{\max}}{q_{\max}} = \frac{2.12}{2^{2-1} - 1} = 2.12$$

1	0	1	0
0	0	-1	1
0	1	0	0
1	0	1	1

Quantized

2.12	0	2.12	0
0	0	-2.12	2.12
0	2.12	0	0
2.12	0	2.12	2.12

Reconstructed

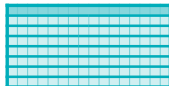
$$\|W - Sq_W\|_F = 2.28$$

Quantization Granularity

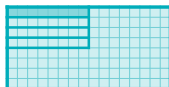
- Per-Tensor Quantization



- Per-Channel Quantization



- **Group Quantization**



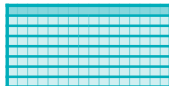
- **Per-Vector Quantization**
- **Shared Micro-exponent (MX) data type**

Quantization Granularity

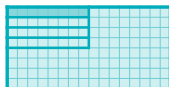
- Per-Tensor Quantization



- Per-Channel Quantization



- **Group Quantization**



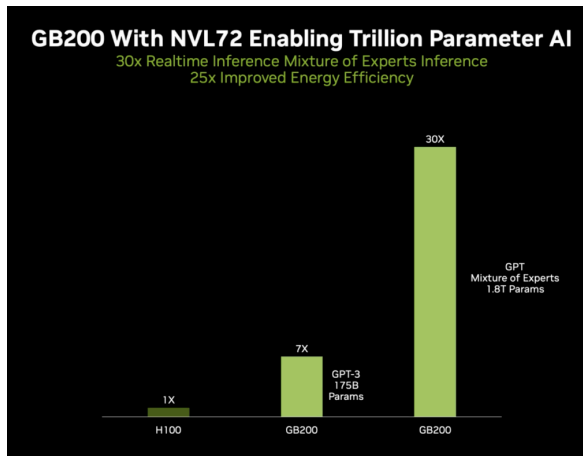
- **Per-Vector Quantization**
- **Shared Micro-exponent (MX) data type**

Why do we need group quantization?

Group Quantization

Achieve a balance between quantization accuracy and hardware efficiency

- Blackwell GPUs support “**micro-tensor scaling**” to optimize accuracy for FP4 AI.
- FP4 tensor core provides 2x higher theoretical throughputs than FP8/FP6/INT8 tensor core.

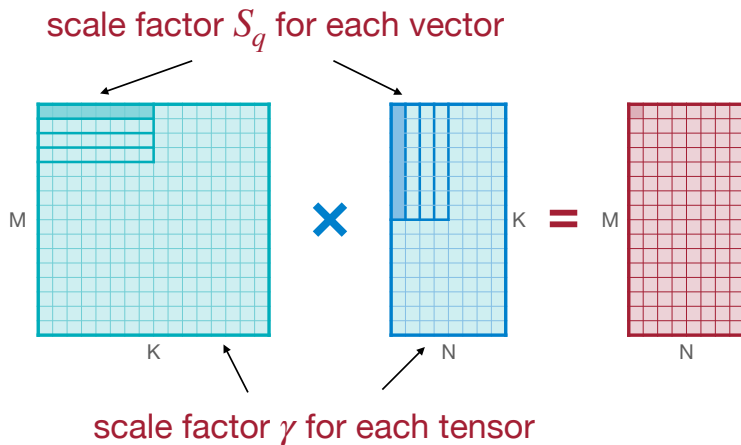


	HGX B200	HGX B100
Blackwell GPUs	8	8
FP4 Tensor Core	144 PetaFLOPS	112 PetaFLOPS
FP8/FP6/INT8	72 PetaFLOPS	56 PetaFLOPS
Fast Memory	Up to 1.5 TB	Up to 1.5TB
Aggregate Memory Bandwidth	Up to 64 TB/s	Up to 64 TB/s
Aggregate NVLink Bandwidth	14.4 TB/s	14.4 TB/s
Per Blackwell GPU Specifications		
FP4 Tensor Core	18 petaFLOPS	14 petaFLOPS
FP8/FP6 Tensor Core	9 petaFLOPS	7 petaFLOPS
INT8 Tensor Core	9 petaOPS	7 petaOPS
FP16/BF16 Tensor Core	4.5 petaFLOPS	3.5 petaFLOPS

VS-Quant: Per-vector Scaled Quantization

Hierarchical scaling factor

- $r = S(q - Z) \rightarrow r = \gamma \cdot S_q(q - Z)$
 - γ is a floating-point coarse grained scale factor
 - S_q is an integer per-vector scale factor
 - achieves a balance between accuracy and hardware efficiency by
 - less expensive integer scale factors at finer granularity
 - more expensive floating-point scale factors at coarser granularity
- Memory Overhead of two-level scaling:
 - Given 4-bit quantization with 4-bit per-vector scale for every 16 elements, the effective bit width is $4 + 4 / 16 = 4.25$ bits.



Group Quantization

Multi-level scaling scheme

	w_{11}								

$$r = (q - z) \cdot s \rightarrow$$

$$r = (q - z) \cdot s_{l_0} \cdot s_{l_1} \cdot \dots$$

r : real number value

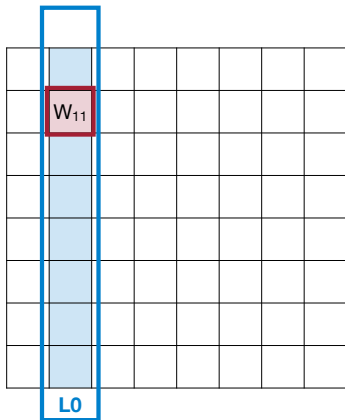
q : quantized value

z : zero point ($z = 0$ is symmetric quantization)

s : scale factors of different levels

Group Quantization

Multi-level scaling scheme



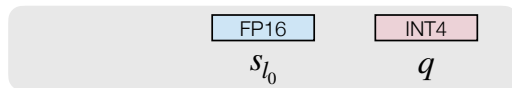
$$r = (q - z) \cdot s_{l_0} \cdot s_{l_1} \cdot \dots$$

r : real number value

q : quantized value

z : zero point ($z = 0$ is symmetric quantization)

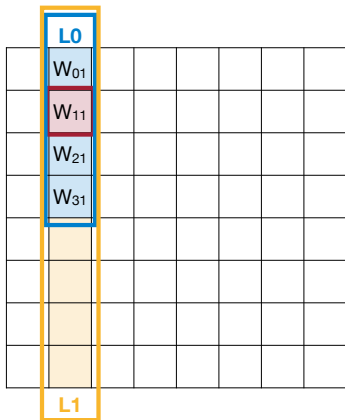
s : scale factors of different levels



Quantization Approach	Data Type	L0 Group Size	L0 Scale Data Type	L1 Group Size	L1 Scale Data Type	Effective Bit Width
Per-Channel Quant	INT4	Per Channel	FP16	-	-	4

Group Quantization

Multi-level scaling scheme



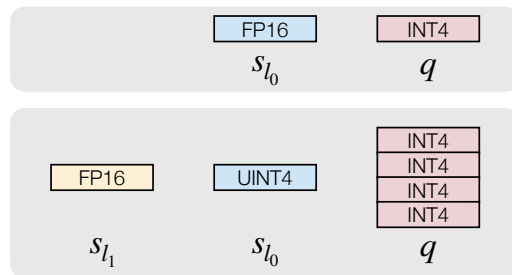
$$r = (q - z) \cdot s_{l_0} \cdot s_{l_1} \cdot \dots$$

r : real number value

q : quantized value

z : zero point ($z = 0$ is symmetric quantization)

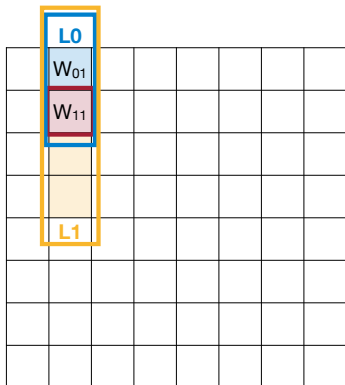
s : scale factors of different levels



Quantization Approach	Data Type	L0 Group Size	L0 Scale Data Type	L1 Group Size	L1 Scale Data Type	Effective Bit Width
Per-Channel Quant	INT4	Per Channel	FP16	-	-	4
VSQ	INT4	16	UINT4	Per Channel	FP16	$4 + 4/16 = 4.25$

Group Quantization

Multi-level scaling scheme



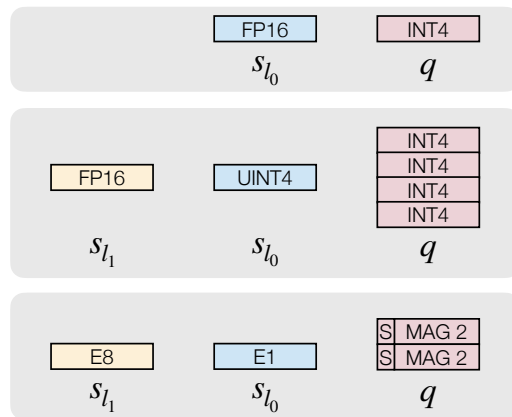
$$r = (q - z) \cdot s_{l_0} \cdot s_{l_1} \cdot \dots$$

r : real number value

q : quantized value

z : zero point ($z = 0$ is symmetric quantization)

s : scale factors of different levels



Quantization Approach	Data Type	L0 Group Size	L0 Scale Data Type	L1 Group Size	L1 Scale Data Type	Effective Bit Width
Per-Channel Quant	INT4	Per Channel	FP16	-	-	4
VSQ	INT4	16	UINT4	Per Channel	FP16	$4 + 4/16 = 4.25$
MX4	S1M2	2	E1M0	16	E8M0	$3 + 1/2 + 8/16 = 4$
MX6	S1M4	2	E1M0	16	E8M0	$5 + 1/2 + 8/16 = 6$
MX9	S1M7	2	E1M0	16	E8M0	$8 + 1/2 + 8/16 = 9$

With Shared Microexponents, A Little Shifting Goes a Long Way [Bita Rouhani *et al.*]

Post-Training Quantization

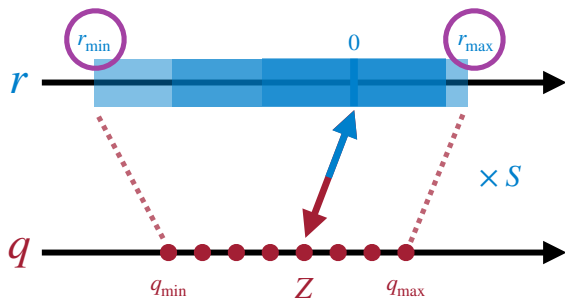
How should we get the optimal linear quantization parameters (S, Z)?

Topic I: Quantization Granularity

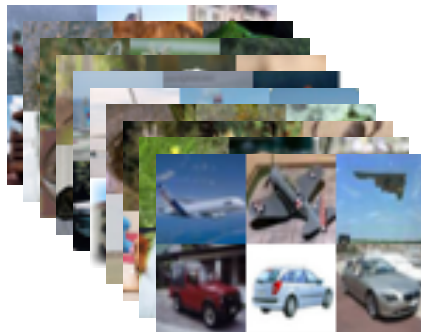
Topic II: Dynamic Range Clipping

Topic III: Rounding

Linear Quantization on Activations



- Unlike weights, the activation range varies across inputs.
- To determine the floating-point range, the activations statistics are gathered **before** deploying the model.

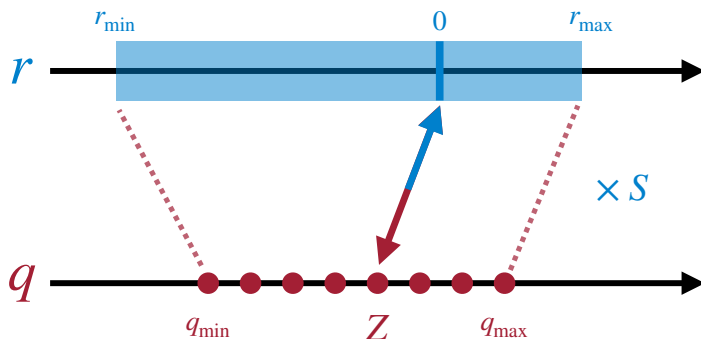


Dynamic Range for Activation Quantization

Collect activations statistics before deploying the model

$$\hat{r}_{\max, \min}^{(t)} = \alpha \cdot r_{\max, \min}^{(t)} + (1 - \alpha) \cdot \hat{r}_{\max, \min}^{(t-1)}$$

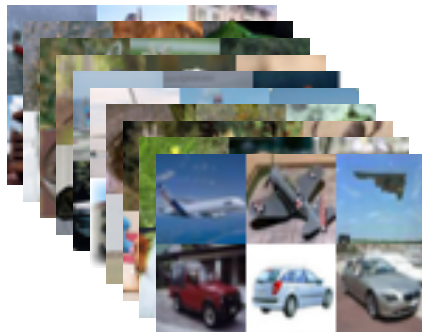
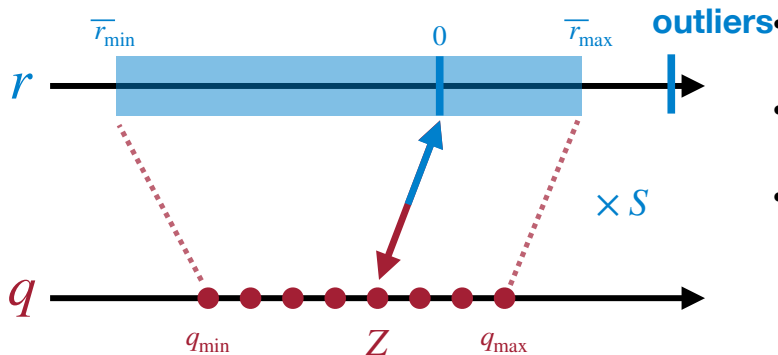
- **Type 1: During training**
 - Exponential moving averages (EMA)
 - observed ranges are smoothed across thousands of training steps



Dynamic Range for Activation Quantization

Collect activations statistics before deploying the model

- **Type 2:** By running a few “**calibration**” batches of samples on the trained FP32 model
- spending dynamic range on the outliers hurts the representation ability.
- use *mean* of the min/max of each sample in the batches
- analytical calculation (see next slide)



Neural Network Distiller

Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]

Dynamic Range for Activation Quantization

Collect activations statistics before deploying the model

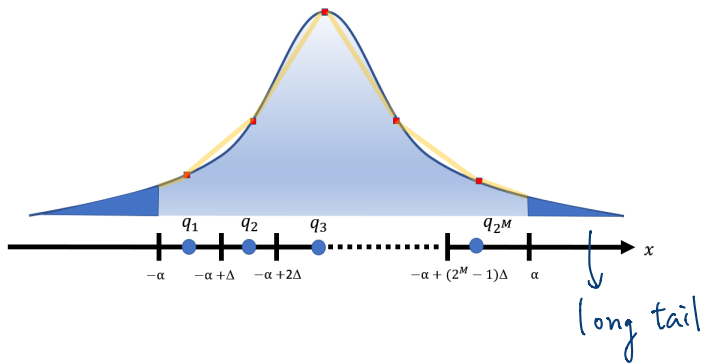
- Type 2: By running a few “calibration” batches of samples on the trained FP32 model
- minimize the mean-square-error between inputs $\mathbf{X}$ and (reconstructed) quantized inputs $Q(\mathbf{X})$,

$$\min_{|r|_{\max}} \mathbb{E} [(\mathbf{X} - Q(\mathbf{X}))^2]$$

- assume inputs are in a **Gaussian or Laplace distribution**. For Laplace $(0, b)$ distribution, optimal clipping values can be solved numerically as

$$|r|_{\max} = 2.83b, 3.89b, 5.03b \text{ for } 2, 3, 4 \text{ bits.}$$

- the Laplace parameter b can be estimated from calibration input distribution.



Dynamic Range for Activation Quantization

Collect activations statistics before deploying the model

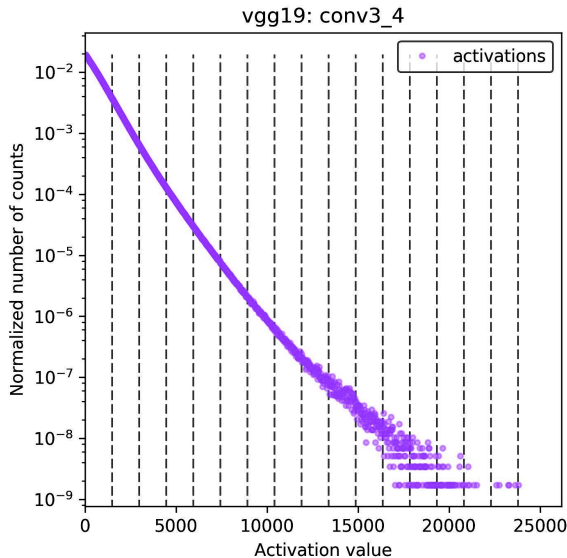
- Type 2: By running a few “calibration” batches of samples on the trained FP32 model
- minimize loss of information, since integer model encodes the same information as the original floating-point model.

- loss of information is measured by **Kullback-Leibler divergence** (relative entropy or information divergence):

- for two discrete probability distributions P, Q

$$D_{KL}(P\|Q) = \sum_i P(x_i) \log \frac{P(x_i)}{Q(x_i)}$$

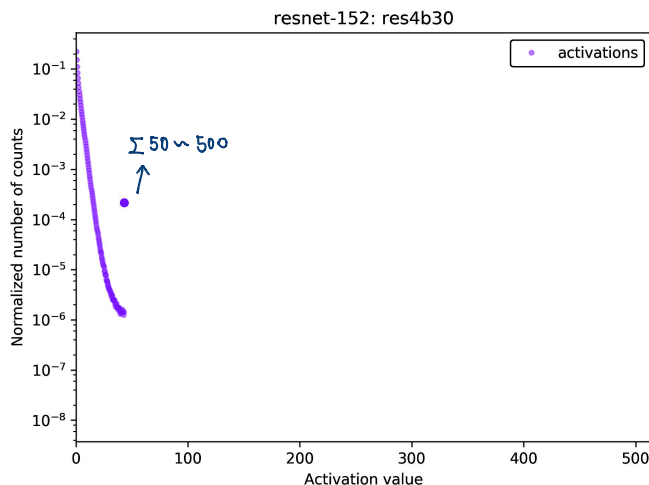
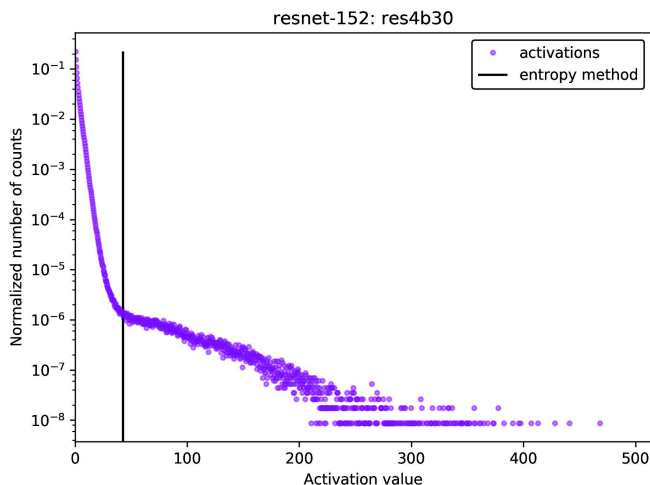
- intuition: KL divergence measures the amount of information lost when approximating a given encoding.



透過 minimize
KL-divergence 來
找到 dynamic range

Dynamic Range for Activation Quantization

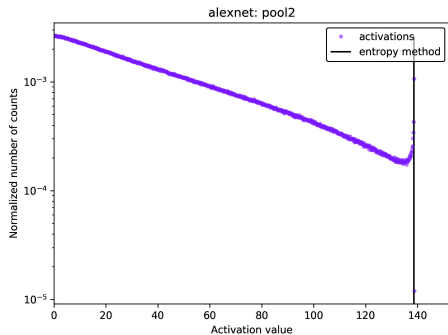
Minimize loss of information by minimizing the KL divergence



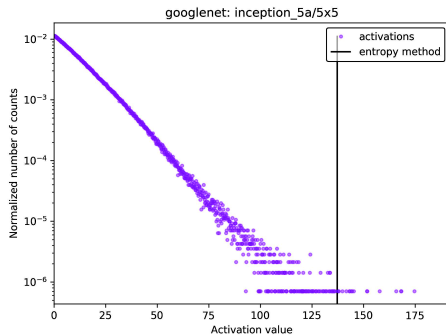
Dynamic Range for Activation Quantization

Minimize loss of information by minimizing the KL divergence

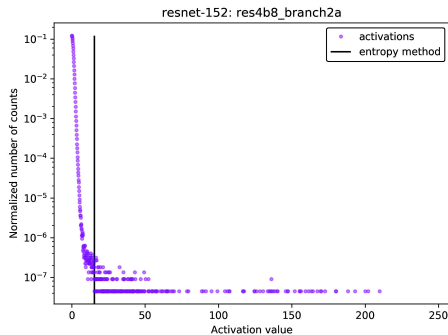
AlexNet: Pool 2



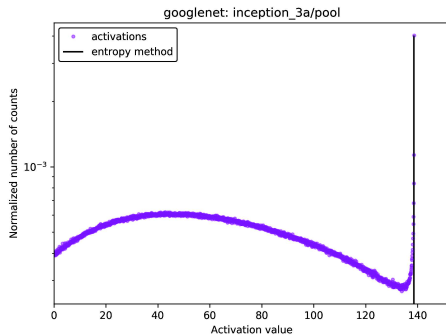
GoogleNet: inception_5a/5x5



ResNet-152: res4b8_branch2a



GoogleNet: inception_3a/pool

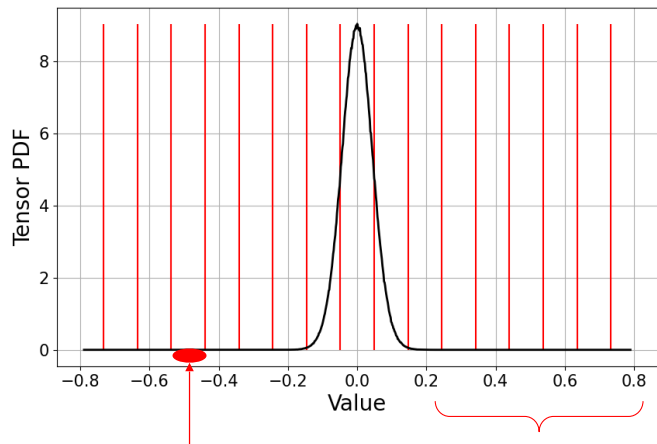


8-bit Inference with TensorRT [Szymon Migacz, 2017]

Dynamic Range for Quantization

Minimize mean-square-error (MSE) using Newton-Raphson method

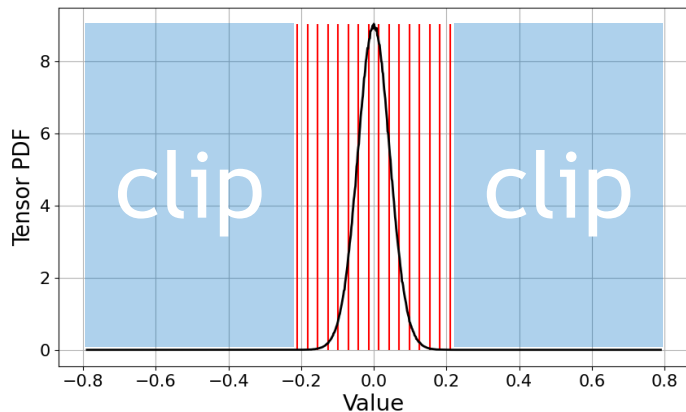
max-scaled quantization



large quantization noise

low density data

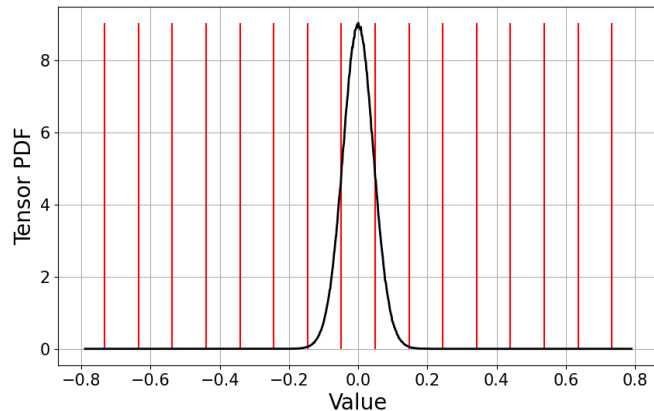
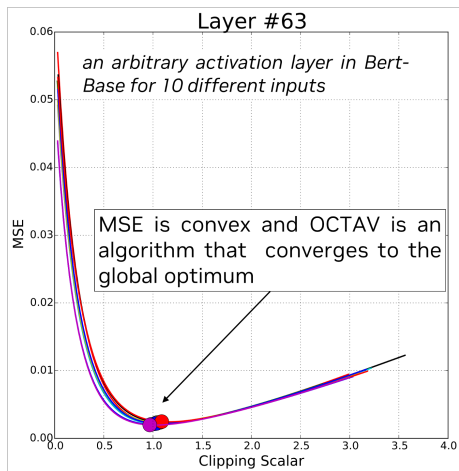
clipped quantization



Optimal Clipping and Magnitude-aware Differentiation for Improved Quantization-aware Training [Sakr et al., ICML 2022]

Dynamic Range for Quantization

Minimize mean-square-error (MSE) using Newton-Raphson method



Network	FP32 Accuracy	OCTAV int4
ResNet-50	76.07	75.84
MobileNet-V2	71.71	70.88
Bert-Large	91.00	87.09

Optimal Clipping and Magnitude-aware Differentiation for Improved Quantization-aware Training [Sakr *et al.*, ICML 2022]

Post-Training Quantization

How should we get the optimal linear quantization parameters (S, Z)?

Topic I: Quantization Granularity

Topic II: Dynamic Range Clipping

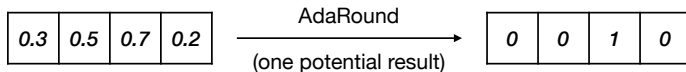
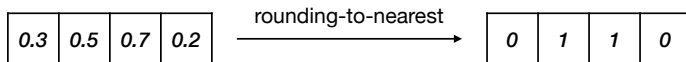
Topic III: Rounding

Adaptive Rounding for Weight Quantization

Rounding-to-nearest is not optimal

- **Philosophy**

- Rounding-to-nearest is not optimal
- Weights are correlated with each other. The best rounding for each weight (to nearest) is not the best rounding for the whole tensor



- What is optimal? Rounding that reconstructs the original activation the best, which may be very different
 - For weight quantization only
 - With short-term tuning, (almost) post-training quantization

Adaptive Rounding for Weight Quantization

Rounding-to-nearest is not optimal

- **Method:**

- Instead of $\lfloor w \rfloor$, we want to choose from $\{ \lfloor w \rfloor, \lceil w \rceil \}$ to get the best reconstruction
- We took a learning-based method to find quantized value $\tilde{w} = \lfloor \lfloor w \rfloor + \delta \rfloor, \delta \in [0,1]$

Adaptive Rounding for Weight Quantization

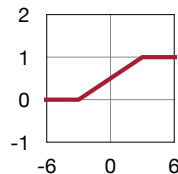
Rounding-to-nearest is not optimal

- **Method:**

- Instead of $\lfloor w \rfloor$, we want to choose from $\{\lfloor w \rfloor, \lceil w \rceil\}$ to get the best reconstruction
- We took a learning-based method to find quantized value $\tilde{w} = \lfloor \lfloor w \rfloor + \delta \rfloor, \delta \in [0,1]$
- We optimize the following equation (omit the derivation): 在四捨五入過程中加入一些隨機性

$$\begin{aligned} & \operatorname{argmin}_{\mathbf{V}} \|\mathbf{W}\mathbf{x} - \tilde{\mathbf{W}}\mathbf{x}\|_F^2 + \lambda f_{reg}(\mathbf{V}) \\ \rightarrow & \operatorname{argmin}_{\mathbf{V}} \|\mathbf{W}\mathbf{x} - \lfloor \lfloor \mathbf{W} \rfloor + \mathbf{h}(\mathbf{V}) \rfloor \mathbf{x}\|_F^2 + \lambda f_{reg}(\mathbf{V}) \end{aligned}$$

- $\mathbf{x}$ is the input to the layer, $\mathbf{V}$ is a random variable of the same shape
- $\mathbf{h}()$ is a function to map the range to $(0,1)$, such as rectified sigmoid
- $f_{reg}(\mathbf{V})$ is a regularization that encourages $\mathbf{h}(\mathbf{V})$ to be binary



Neural Network Quantization

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	3:	2.00
1	1	0	3	2:	1.50
0	3	1	0	1:	0.00
3	1	2	2	0:	-1.00

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

$(-1) \times 1.07$

K-Means-based Quantization

Linear Quantization

Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic

- **Zero Point**
 - Asymmetric
 - Symmetric
- **Scaling Granularity**
 - Per-Tensor
 - Per-Channel
 - Group Quantization
- **Range Clipping**
 - Exponential Moving Average
 - Minimizing KL Divergence
 - Minimizing Mean-Square-Error
- **Rounding**
 - Round-to-Nearest
 - AdaRound

Post-Training INT8 Linear Quantization

Activation		Symmetric	Asymmetric
		Per-Tensor	Per-Tensor
		Minimize KL-Divergence	Exponential Moving Average (EMA)
Weight		Symmetric	Symmetric
		Per-Tensor	Per-Channel
Neural Network	GoogLeNet	-0.45%	0%
	ResNet-50	-0.13%	-0.6%
	ResNet-152	-0.08%	-1.8%
	MobileNetV1	-	-11.8%
	MobileNetV2	-	-2.1%

Data-Free Quantization Through Weight Equalization and Bias Correction [Markus *et al.*, ICCV 2019]
Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper [Raghuraman Krishnamoorthi, arXiv 2018]
8-bit Inference with TensorRT [Szymon Migacz, 2017]

Post-Training INT8 Linear Quantization

Activation		Symmetric	Asymmetric
		Per-Tensor	Per-Tensor
		Minimize KL-Divergence	Exponential Moving Average (EMA)
Weight		Symmetric	Symmetric
		Per-Tensor	Per-Channel
Neural Network	Smaller models seem to not respond as well to post-training quantization, presumably due to their smaller representational capacity.		How should we improve performance of quantized models?
	MobileNetV1	-	-11.8%
	MobileNetV2	-	-2.1%

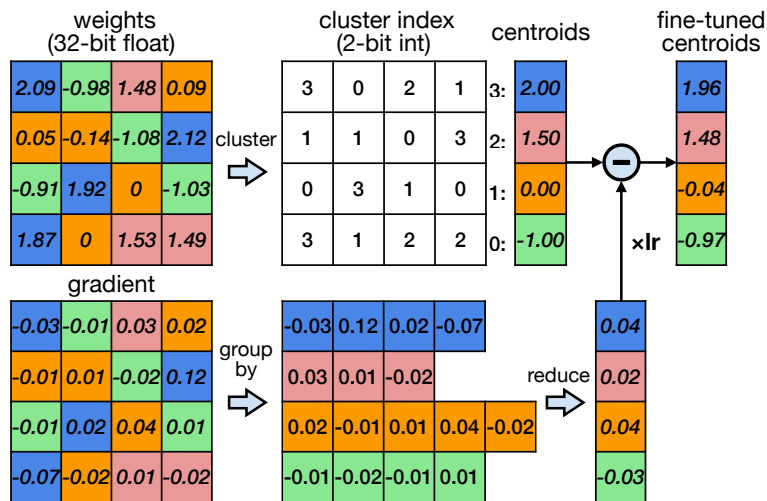
Data-Free Quantization Through Weight Equalization and Bias Correction [Markus *et al.*, ICCV 2019]
Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper [Raghuraman Krishnamoorthi, arXiv 2018]
8-bit Inference with TensorRT [Szymon Migacz, 2017]

Quantization-Aware Training

How should we improve performance of quantized models?

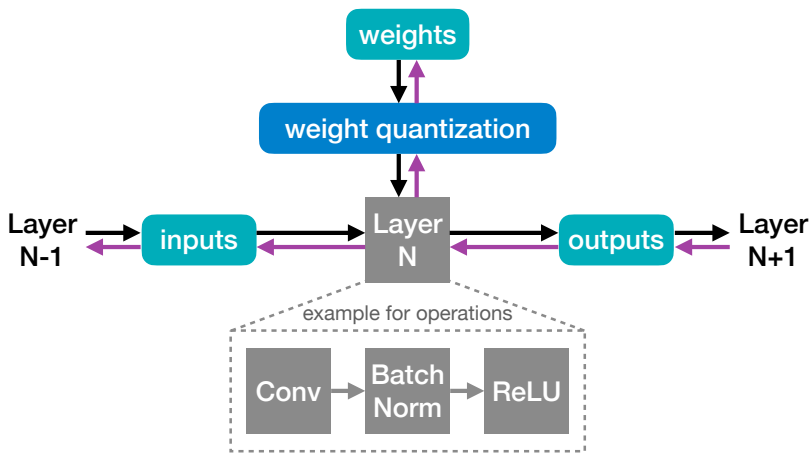
Quantization-Aware Training

- To minimize the loss of accuracy, especially aggressive quantization with 4 bits and lower bit width, neural network will be trained/fine-tuned with quantized weights and activations.
- Usually, fine-tuning a pre-trained floating point model provides better accuracy than training from scratch.



Quantization-Aware Training

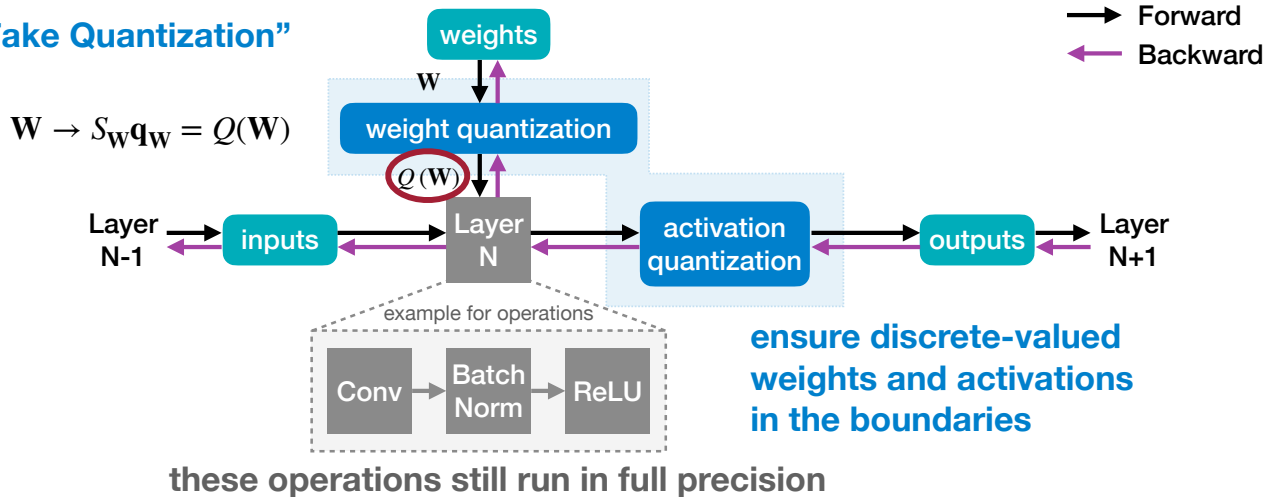
- A full precision copy of the weights W is maintained throughout the training.
- The small gradients are accumulated without loss of precision.
- Once the model is trained, only the quantized weights are used for inference.



Quantization-Aware Training

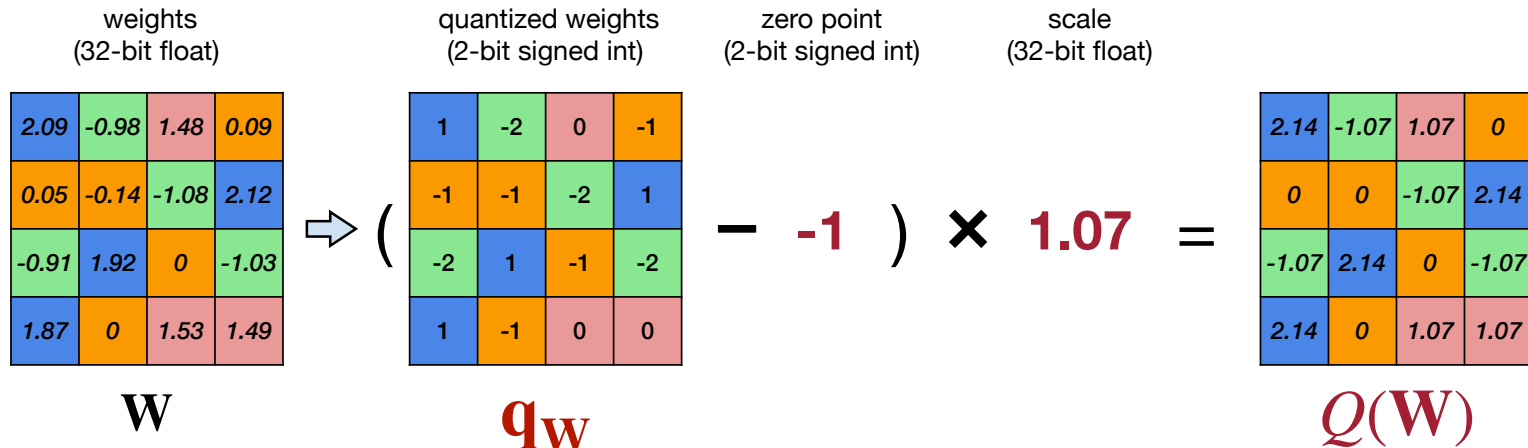
- A full precision copy of the weights W is maintained throughout the training.
- The small gradients are accumulated without loss of precision.
- Once the model is trained, only the quantized weights are used for inference.

“Simulated/Fake Quantization”



Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$

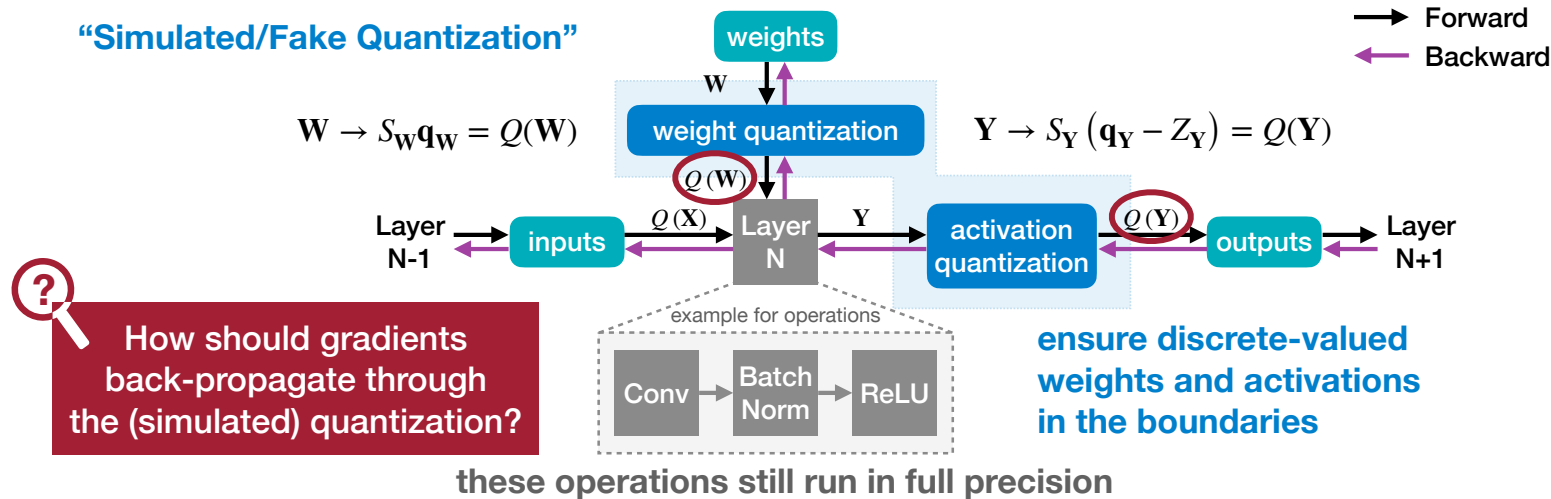


Quantization-Aware Training

Train the model taking quantization into consideration

- A full precision copy of the weights W is maintained throughout the training.
- The small gradients are accumulated without loss of precision.
- Once the model is trained, only the quantized weights are used for inference.

“Simulated/Fake Quantization”



Straight-Through Estimator (STE)

- Quantization is discrete-valued, and thus the derivative is 0 almost everywhere.

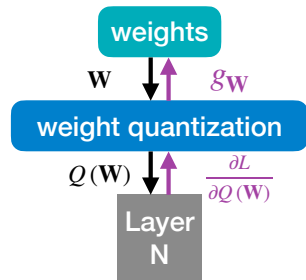
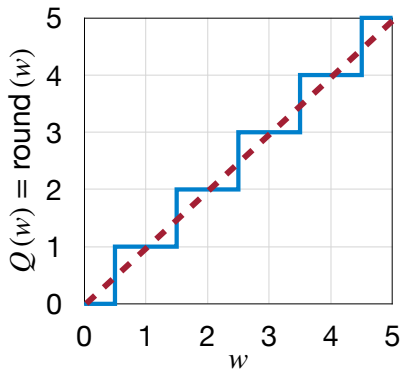
$$\frac{\partial Q(W)}{\partial W} = 0$$

- The neural network will learn nothing since gradients become 0 and the weights won't get updated.

$$g_W = \frac{\partial L}{\partial W} = \frac{\partial L}{\partial Q(W)} \cdot \frac{\partial Q(W)}{\partial W} = 0$$

- Straight-Through Estimator (STE) simply passes the gradients through the quantization as if it had been the *identity* function.

$$g_W = \frac{\partial L}{\partial W} = \frac{\partial L}{\partial Q(W)}$$



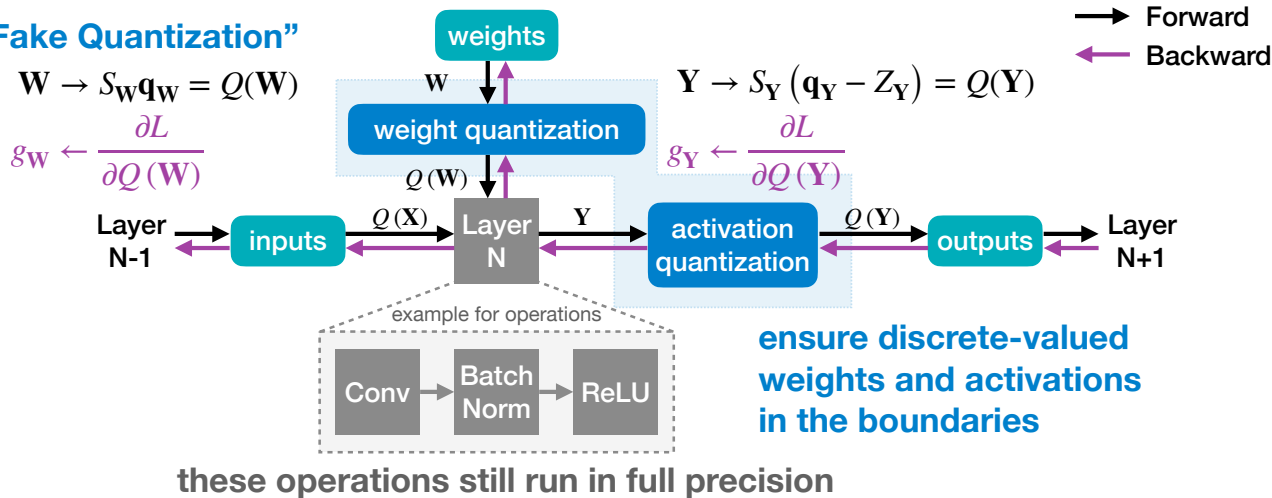
Neural Networks for Machine Learning [Hinton *et al.*, Coursera Video Lecture, 2012]
Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation [Bengio, arXiv 2013]

Quantization-Aware Training

Train the model taking quantization into consideration

- A full precision copy of the weights is maintained throughout the training.
- The small gradients are accumulated without loss of precision.
- Once the model is trained, only the quantized weights are used for inference.

“Simulated/Fake Quantization”



INT8 Linear Quantization-Aware Training

Neural Network	Floating-Point	Post-Training Quantization		Quantization-Aware Training	
		Asymmetric	Symmetric	Asymmetric	Symmetric
		Per-Tensor	Per-Channel	Per-Tensor	Per-Channel
MobileNetV1	70.9%	0.1%	59.1%	70.0%	70.7%
MobileNetV2	71.9%	0.1%	69.8%	70.9%	71.1%
NASNet-Mobile	74.9%	72.2%	72.1%	73.0%	73.0%

Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper [Raghuraman Krishnamoorthi, arXiv 2018]

Neural Network Quantization

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	3:	2.00
1	1	0	3	2:	1.50
0	3	1	0	1:	0.00
3	1	2	2	0:	-1.00

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

$(\dots) - (-1) \times 1.07$

**K-Means-based
Quantization**

**Linear
Quantization**

Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic



Neural Network Quantization

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	3:	2.00
1	1	0	3	2:	1.50
0	3	1	0	1:	0.00
3	1	2	2	0:	-1.00

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

$(\dots) - (-1) \times 1.07$

1	0	1	1
1	0	0	1
0	1	1	0
1	1	1	1

**K-Means-based
Quantization**

**Linear
Quantization**

**Binary/Ternary
Quantization**

Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights	Binary/Ternary Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic	Bit Operations

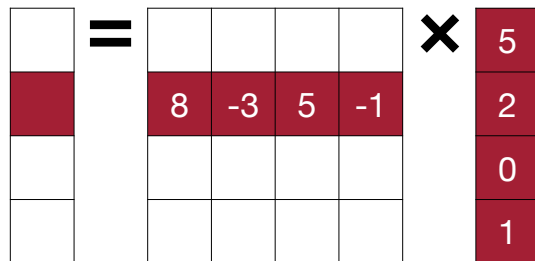
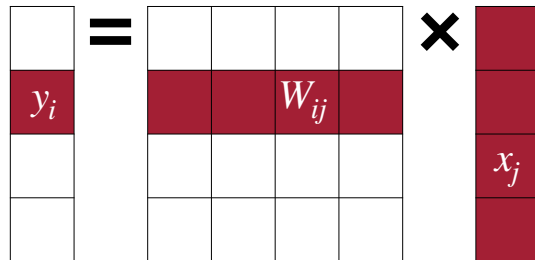
Binary/Ternary Quantization

Can we push the quantization precision to 1 bit?

Can quantization bit width go even lower?

$$y_i = \sum_j W_{ij} \cdot x_j$$
$$= 8 \times 5 + (-3) \times 2 + 5 \times 0 + (-1) \times 1$$

input	weight	operations	memory	computation
$\mathbb{R}$	$\mathbb{R}$	$+ \times$	$1 \times$	$1 \times$



If weights are quantized to +1 and -1

$$y_i = \sum_j W_{ij} \cdot x_j$$

$$= 5 - 2 + 0 - 1$$

$$\begin{bmatrix} \square \\ \blacksquare \\ \square \\ \square \end{bmatrix} = \begin{bmatrix} \square & \square & \square & \square \\ \blacksquare & 8 & -3 & 5 & -1 \\ \square & \square & \square & \square \\ \square & \square & \square & \square \end{bmatrix} \times \begin{bmatrix} \square \\ \blacksquare \\ \square \\ \blacksquare \end{bmatrix} \begin{matrix} 5 \\ 2 \\ 0 \\ 1 \end{matrix}$$

input	weight	operations	memory	computation
$\mathbb{R}$	$\mathbb{R}$	$+ \times$	$1\times$	$1\times$
$\mathbb{R}$	$\mathbb{B}$	$+ -$	$\sim 32\times$ less	$\sim 2\times$ less

$$\begin{bmatrix} \square \\ \blacksquare \\ \square \\ \square \end{bmatrix} = \begin{bmatrix} \square & \square & \square & \square \\ \blacksquare & 1 & -1 & 1 & -1 \\ \square & \square & \square & \square \\ \square & \square & \square & \square \end{bmatrix} \times \begin{bmatrix} \square \\ \blacksquare \\ \square \\ \blacksquare \end{bmatrix} \begin{matrix} 5 \\ 2 \\ 0 \\ 1 \end{matrix}$$

BinaryConnect: Training Deep Neural Networks with Binary Weights during Propagations [Courbariaux *et al.*, NeurIPS 2015]
 XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

Binarization

- **Deterministic Binarization**

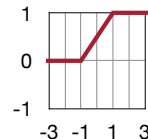
- directly computes the bit value based on a threshold, usually 0, resulting in a sign function.

$$q = \text{sign}(r) = \begin{cases} +1, & r \geq 0 \\ -1, & r < 0 \end{cases}$$

- **Stochastic Binarization**

- use global statistics or the value of input data to determine the probability of being -1 or +1
 - e.g., in Binary Connect (BC), probability is determined by hard sigmoid function $\sigma(r)$

$$q = \begin{cases} +1, & \text{with probability } p = \sigma(r) \\ -1, & \text{with probability } 1 - p \end{cases}, \quad \text{where } \sigma(r) = \min(\max(\frac{r+1}{2}, 0), 1)$$



- harder to implement as it requires the hardware to generate random bits when quantizing.

BinaryConnect: Training Deep Neural Networks with Binary Weights during Propagations [Courbariaux *et al.*, NeurIPS 2015]
BinaryNet: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1. [Courbariaux *et al.*, Arxiv 2016]

Minimizing Quantization Error in Binarization

weights (32-bit float)			
2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

$\mathbf{W} \rightarrow \mathbf{W}^{\mathbb{B}}$

binary weights (1-bit)			
1	-1	1	1
1	-1	-1	1
-1	1	1	-1
1	1	1	1

$$\mathbf{W}^{\mathbb{B}} = \text{sign}(\mathbf{W})$$

$$\alpha = \frac{1}{n} \|\mathbf{W}\|_1$$

$\alpha \mathbf{W}^{\mathbb{B}}$

1	-1	1	1
1	-1	-1	1
-1	1	1	-1
1	1	1	1

$$\|\mathbf{W} - \mathbf{W}^{\mathbb{B}}\|_F^2 = 9.28$$

scale
(32-bit float)

$$\times 1.05 = \frac{1}{16} \|\mathbf{W}\|_1$$

$$\|\mathbf{W} - \alpha \mathbf{W}^{\mathbb{B}}\|_F^2 = 9.24$$

AlexNet-based Network	ImageNet Top-1 Accuracy Delta
BinaryConnect	-21.2%
Binary Weight Network (BWN)	0.2%

If both activations and weights are binarized

$$\begin{aligned}y_i &= \sum_j W_{ij} \cdot x_j \\&= 1 \times 1 + (-1) \times 1 + 1 \times (-1) + (-1) \times 1 \\&= 1 + (-1) + (-1) + (-1) = -2\end{aligned}$$

A diagram illustrating a dot product operation. On the left is a 4x1 column vector with one red cell (value 1) at the second row. This is followed by an equals sign, then a 4x4 matrix with one red row (values 8, -3, 5, -1) at the second row. This is followed by a multiplication symbol (X), then another 4x1 column vector with one red cell (value 0) at the third row.

A diagram illustrating a dot product operation. On the left is a 4x1 column vector with one red cell (value 1) at the second row. This is followed by an equals sign, then a 4x4 matrix with one red row (values 1, -1, 1, -1) at the second row. This is followed by a multiplication symbol (X), then another 4x1 column vector with one red cell (value -1) at the third row.

If both activations and weights are binarized

$$y_i = \sum_j W_{ij} \cdot x_j$$
$$= 1 \times 1 + (-1) \times 1 + 1 \times (-1) + (-1) \times 1$$
$$= 1 + (-1) + (-1) + (-1) = -2$$

W	X	Y=WX
1	1	1
1	-1	-1
-1	-1	1
-1	1	-1

bw	bx	XNOR(bw, bx)
1	1	1
1	0	0
0	0	1
0	1	0

XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

If both activations and weights are binarized

$$y_i = \sum_j W_{ij} \cdot x_j$$

$$= 1 \times 1 + (-1) \times 1 + 1 \times (-1) + (-1) \times 1$$

$$= 1 + (-1) + (-1) + (-1) = -2$$

$$= 1 \text{ xnor } 1 + 0 \text{ xnor } 1 + 1 \text{ xnor } 0 + 0 \text{ xnor } 1$$

$$= 1 + 0 + 0 + 0 = 1$$

↑ ?

W	X	Y=WX
1	1	1
1	-1	-1
-1	-1	1
-1	1	-1

bw	bx	XNOR(bw, bx)
1	1	1
1	0	0
0	0	1
0	1	0

XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

If both activations and weights are binarized

$$y_i = \sum_j W_{ij} \cdot x_j$$

$$= 1 \times 1 + (-1) \times 1 + 1 \times (-1) + (-1) \times 1$$

$$= 1 + (-1) + (-1) + (-1) = -2$$

$$y_i = -n + 2 \cdot \sum_j W_{ij} \text{ xnor } x_j$$

$$= 1 \text{ xnor } 1 + 0 \text{ xnor } 1 + 1 \text{ xnor } 0 + 0 \text{ xnor } 1$$

$$= 1 + 0 + 0 + 0 = \boxed{1 \times 2 + -4} = -2$$

Assuming $\begin{matrix} \uparrow +2 \\ -1 & -1 & -1 & -1 \end{matrix} \rightarrow -4$

W	X	Y=WX
1	1	1
1	-1	-1
-1	-1	1
-1	1	-1

bw	bx	XNOR(bw, bx)
1	1	1
1	0	0
0	0	1
0	1	0

XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

If both activations and weights are binarized

$$y_i = -n + 2 \cdot \sum_j W_{ij} \text{ xnor } x_j \quad \rightarrow \quad y_i = -n + \text{popcount}(W_i \text{ xnor } x) \ll 1$$

$$= -4 + 2 \times (1 \text{ xnor } 1 + 0 \text{ xnor } 1 + 1 \text{ xnor } 0 + 0 \text{ xnor } 1)$$

$$= -4 + 2 \times (1 + 0 + 0 + 0) = -2$$

→ **popcount**: return the number of 1

W	X	Y=WX
1	1	1
1	-1	-1
-1	-1	1
-1	1	-1

bw	bx	XNOR(bw, bx)
1	1	1
1	0	0
0	0	1
0	1	0

XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

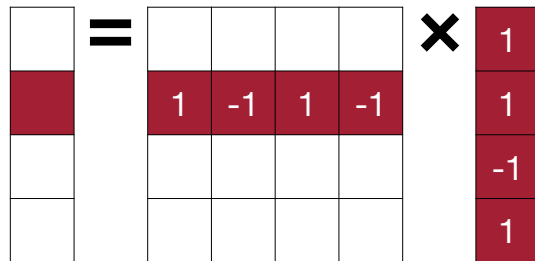
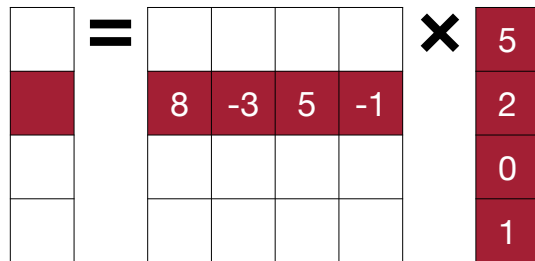
If both activations and weights are binarized

$$y_i = -n + \text{popcount}(W_i \text{ xnor } x) \ll 1$$

$$= -4 + \text{popcount}(1010 \text{ xnor } 1101) \ll 1$$

$$= -4 + \text{popcount}(1000) \ll 1 = -4 + 2 = -2$$

input	weight	operations	memory	computation
$\mathbb{R}$	$\mathbb{R}$	$+ \times$	$1\times$	$1\times$
$\mathbb{R}$	$\mathbb{B}$	$+ -$	$\sim 32\times$ less	$\sim 2\times$ less
$\mathbb{B}$	$\mathbb{B}$	xnor, popcount	$\sim 32\times$ less	$\sim 58\times$ less



XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

Accuracy Degradation of Binarization

Neural Network	Quantization	Bit-Width		ImageNet Top-1 Accuracy Delta
		W	A	
AlexNet	BWN	1	32	0.2%
	BNN	1	1	-28.7%
	XNOR-Net	1	1	-12.4%

- * BWN: Binary Weight Network with scale for weight binarization
- * BNN: Binarized Neural Network without scale factors
- * XNOR-Net: scale factors for both activation and weight binarization

Binarized Neural Networks: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1. [Courbariaux *et al.*, Arxiv 2016]
XNOR-Net: ImageNet Classification using Binary Convolutional Neural Networks [Rastegari *et al.*, ECCV 2016]

Ternary Weight Networks (TWN)

Weights are quantized to +1, -1 and 0

$$q = \begin{cases} r_t, & r > \Delta \\ 0, & |r| \leq \Delta, \\ -r_t, & r < -\Delta \end{cases} \quad \text{where } \Delta = 0.7 \times \mathbb{E}(|r|), r_t = \mathbb{E}_{|r| > \Delta}(|r|)$$

weights $\mathbf{W}$
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49



ternary weights $\mathbf{W}^\top$
(2-bit)

1	-1	1	0
0	0	-1	1
-1	1	0	-1
1	0	1	1

$$\Delta = 0.7 \times \frac{1}{16} \|\mathbf{W}\|_1 = 0.73$$

$$\times 1.5 = \frac{1}{11} \|\mathbf{W}_{\mathbf{W}^\top \neq 0}\|_1$$

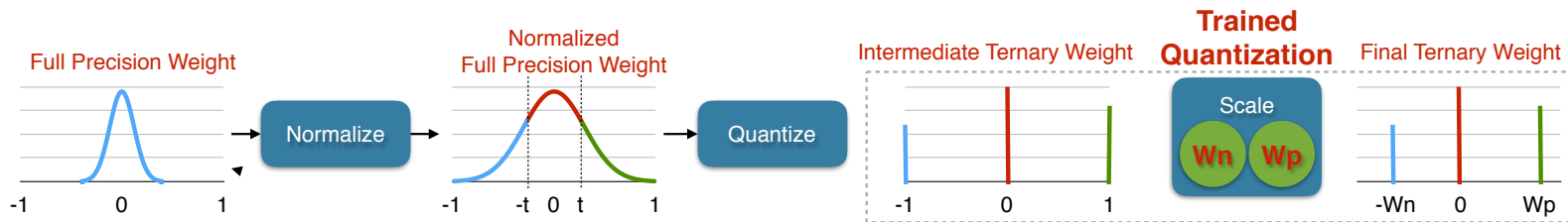
ImageNet Top-1 Accuracy	Full Precision	1 bit (BWN)	2 bit (TWN)
ResNet-18	69.6	60.8	65.3

Ternary Weight Networks [Li et al., Arxiv 2016]

Trained Ternary Quantization (TTQ)

- Instead of using fixed scale r_t , TTQ introduces two *trainable* parameters w_p and w_n to represent the positive and negative scales in the quantization.

$$q = \begin{cases} w_p, & r > \Delta \\ 0, & |r| \leq \Delta \\ -w_n, & r < -\Delta \end{cases}$$



ImageNet Top-1 Accuracy	Full Precision	1 bit (BWN)	2 bit (TWN)	TTQ
ResNet-18	69.6	60.8	65.3	66.6

Trained Ternary Quantization [Zhu et al., ICLR 2017]

Neural Network Quantization

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

3	0	2	1	3:	2.00
1	1	0	3	2:	1.50
0	3	1	0	1:	0.00
3	1	2	2	0:	-1.00

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

$(\dots) - (-1) \times 1.07$

1	0	1	1
1	0	0	1
0	1	1	0
1	1	1	1

**K-Means-based
Quantization**

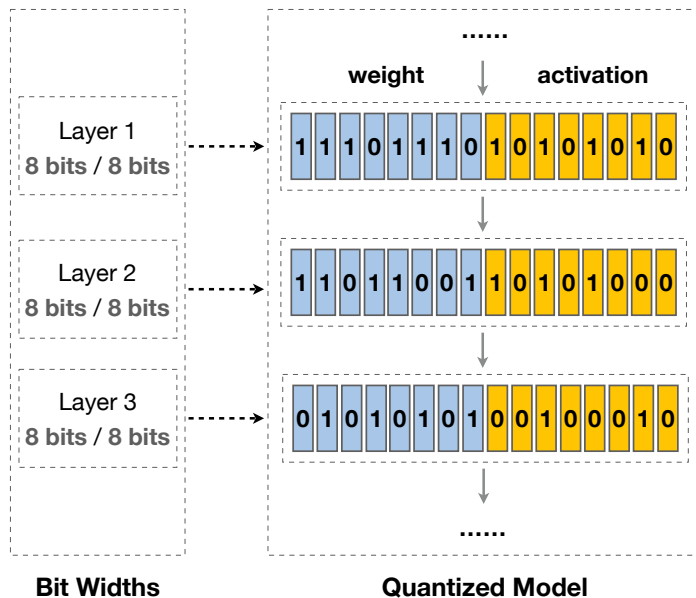
**Linear
Quantization**

**Binary/Ternary
Quantization**

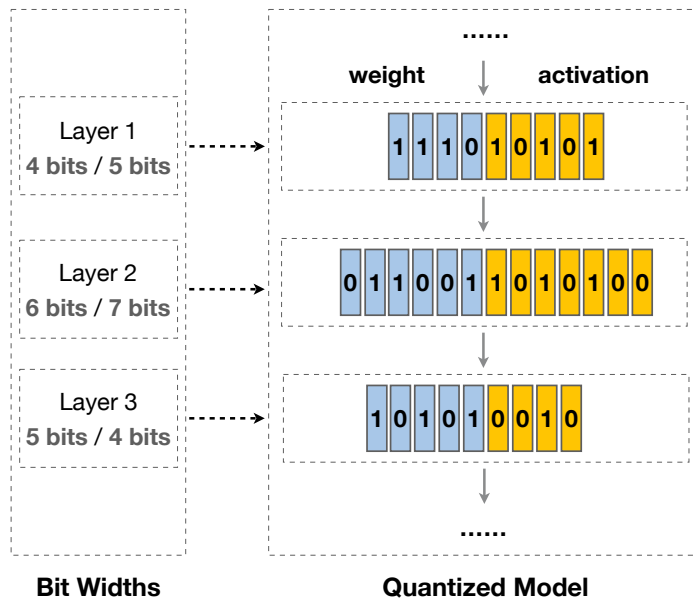
Storage	Floating-Point Weights	Integer Weights; Floating-Point Codebook	Integer Weights	Binary/Ternary Weights
Computation	Floating-Point Arithmetic	Floating-Point Arithmetic	Integer Arithmetic	Bit Operations

Mixed-Precision Quantization

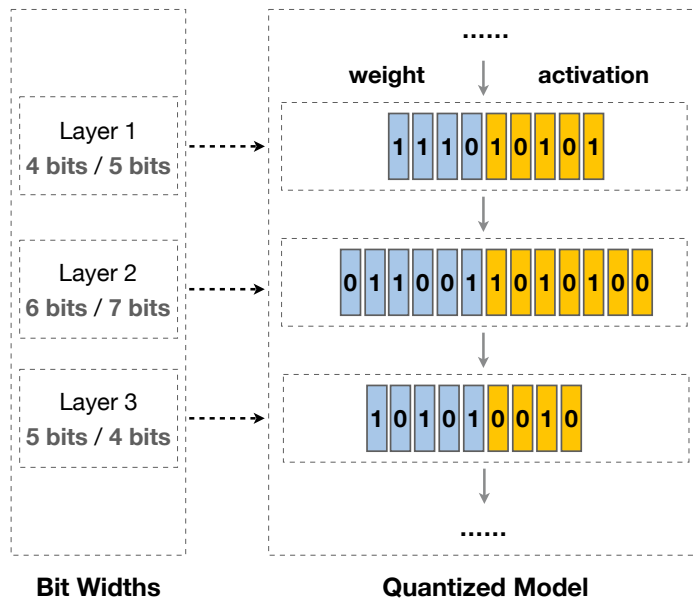
Uniform Quantization



Mixed-Precision Quantization



Challenge: Huge Design Space



Choices: $8 \times 8 = 64$

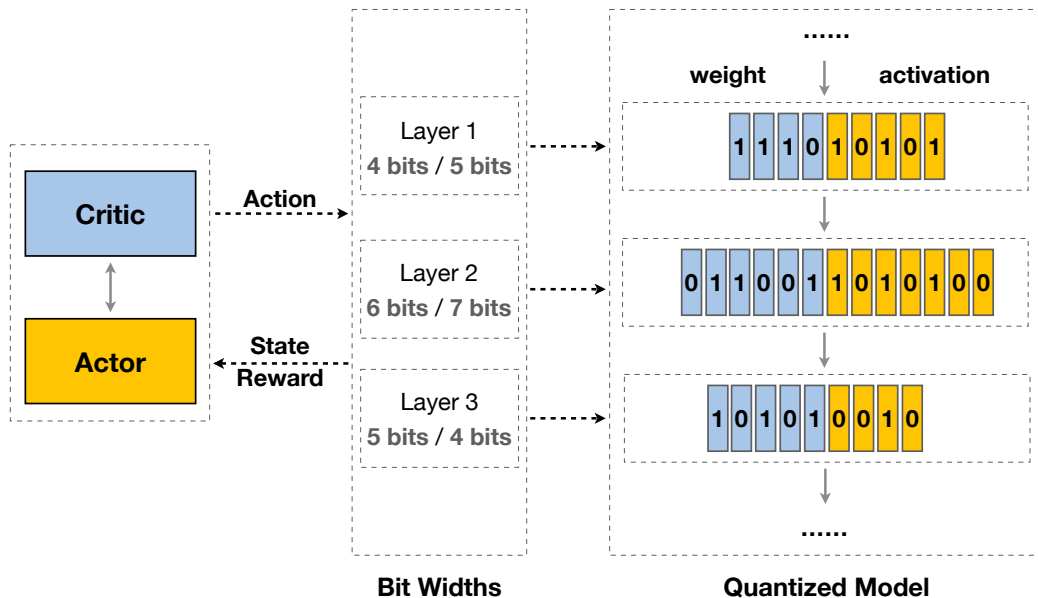
Choices: $8 \times 8 = 64$

Choices: $8 \times 8 = 64$



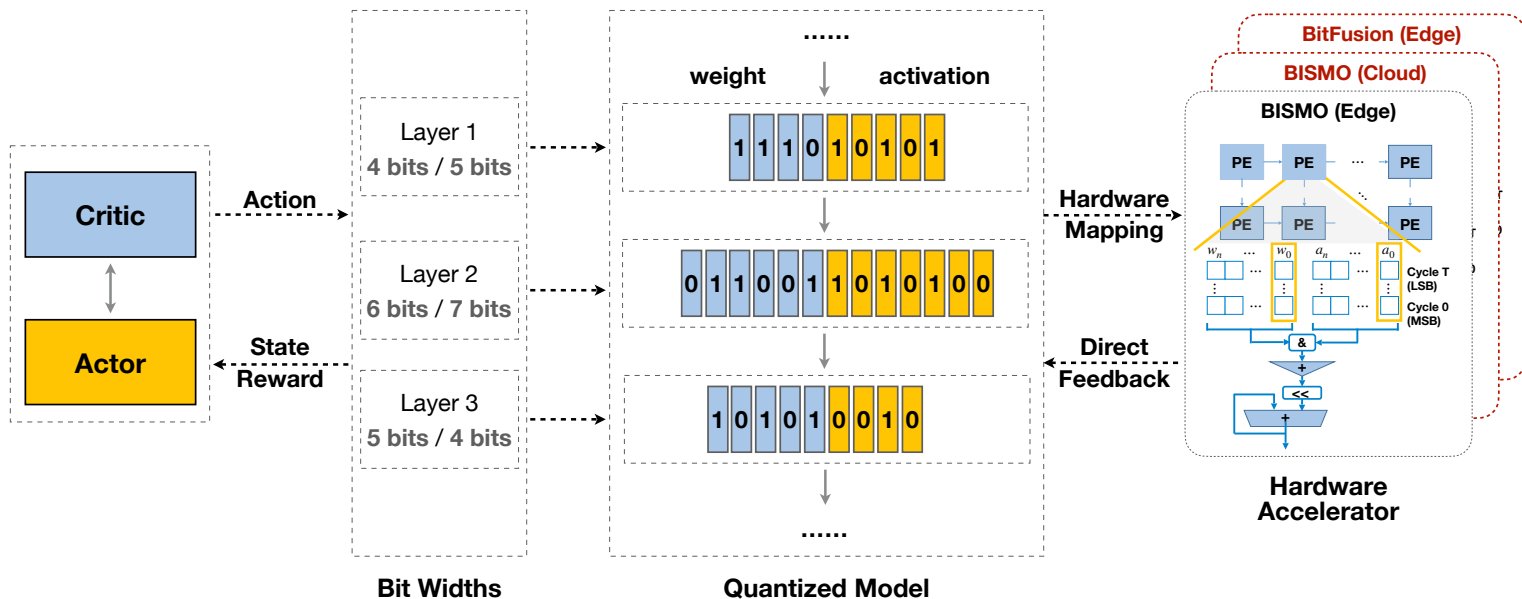
Design Space: 64^n

Solution: Design Automation



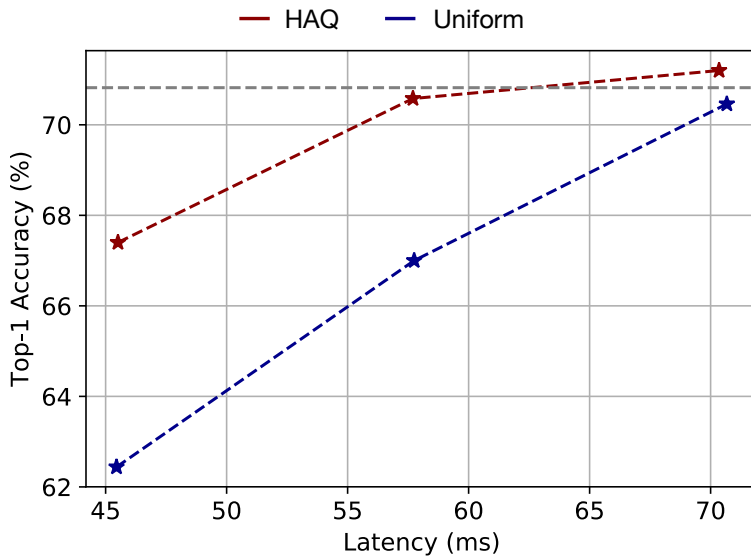
HAQ: Hardware-Aware Automated Quantization with Mixed Precision [Wang *et al.*, CVPR 2019]

Solution: Design Automation



HAQ: Hardware-Aware Automated Quantization with Mixed Precision [Wang et al., CVPR 2019]

HAQ Outperforms Uniform Quantization



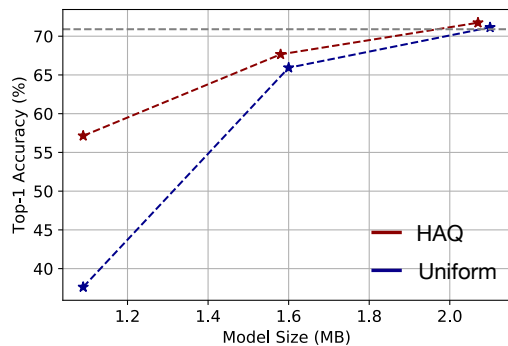
Mixed-Precision Quantized MobileNetV1

HAQ: Hardware-Aware Automated Quantization with Mixed Precision [Wang *et al.*, CVPR 2019]

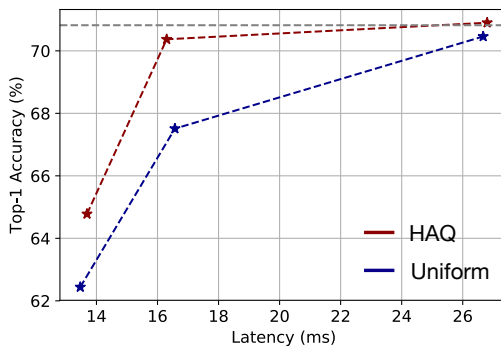
HAQ Supports Multiple Objectives



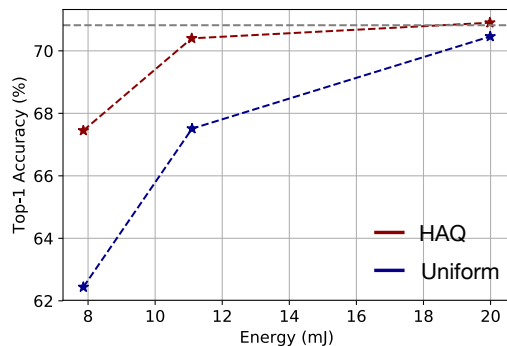
Model Size Constrained



Latency Constrained



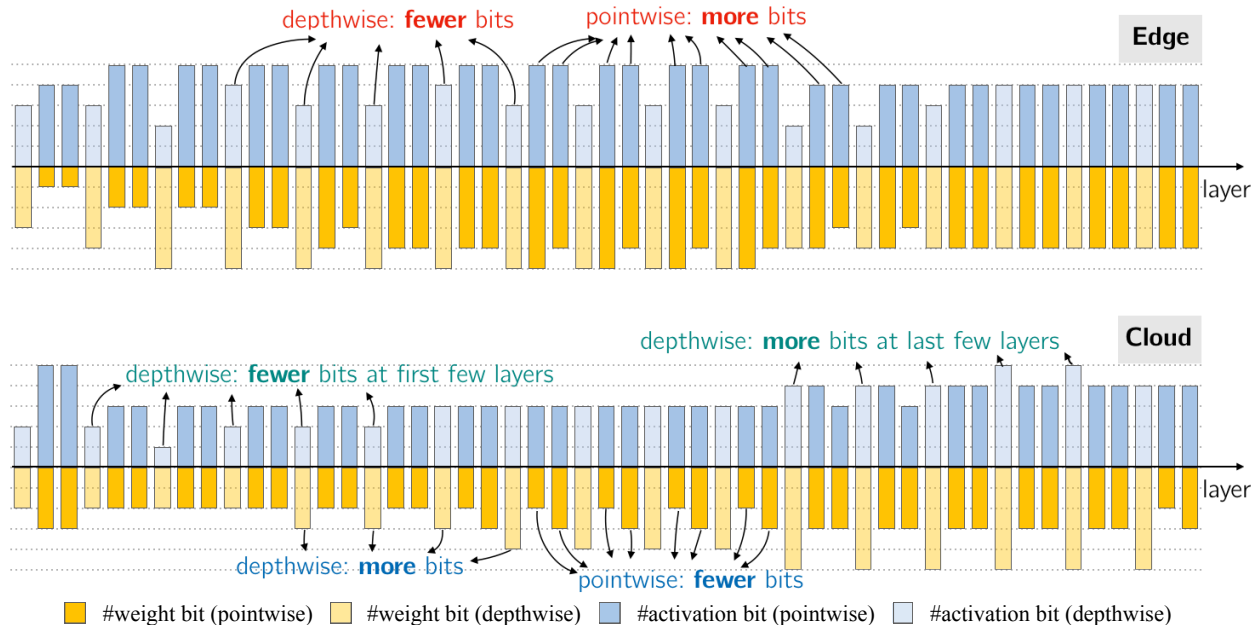
Energy Constrained



Mixed-Precision Quantized MobileNetV1

HAQ: Hardware-Aware Automated Quantization with Mixed Precision [Wang *et al.*, CVPR 2019]

Quantization Policy for Edge and Cloud



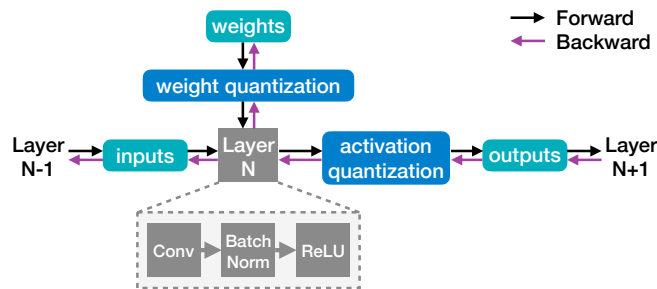
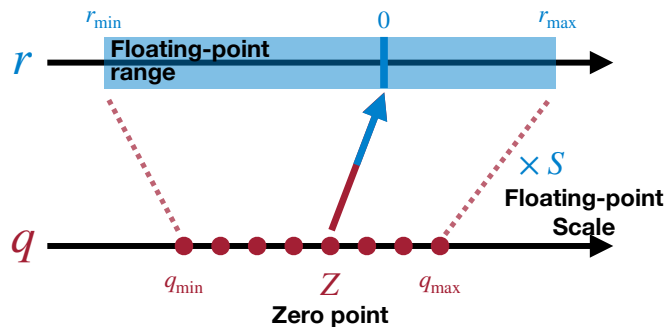
Mixed-Precision Quantized MobileNetV2

HAQ: Hardware-Aware Automated Quantization with Mixed Precision [Wang *et al.*, CVPR 2019]

Summary of Today's Lecture

In this lecture, we

1. Reviewed Linear Quantization.
2. Introduced **Post-Training Quantization (PTQ)** that quantizes an already-trained floating-point neural network model.
 - Per-tensor vs. per-channel vs. group quantization
 - How to determine dynamic range for quantization
3. Introduced **Quantization-Aware Training (QAT)** that emulates inference-time quantization during the training/fine-tuning.
 - Straight-Through Estimator (STE)
4. Introduced **binary and ternary** quantization.
5. Introduced automatic **mixed-precision** quantization.



References

1. Deep Compression [Han et al., ICLR 2016]
2. Neural Network Distiller: https://intellabs.github.io/distiller/algo_quantization.html
3. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference [Jacob et al., CVPR 2018]
4. Data-Free Quantization Through Weight Equalization and Bias Correction [Markus et al., ICCV 2019]
5. Post-Training 4-Bit Quantization of Convolution Networks for Rapid-Deployment [Banner et al., NeurIPS 2019]
6. 8-bit Inference with TensorRT [Szymon Migacz, 2017]
7. Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper [Raghuraman Krishnamoorthi, arXiv 2018]
8. Neural Networks for Machine Learning [Hinton et al., Coursera Video Lecture, 2012]
9. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation [Bengio, arXiv 2013]
10. Binarized Neural Networks: Training Deep Neural Networks with Weights and Activations Constrained to +1 or -1. [Courbariaux et al., Arxiv 2016]
11. DoReFa-Net: Training Low Bitwidth Convolutional Neural Networks with Low Bitwidth Gradients [Zhou et al., arXiv 2016]
12. PACT: Parameterized Clipping Activation for Quantized Neural Networks [Choi et al., arXiv 2018]
13. WRPN: Wide Reduced-Precision Networks [Mishra et al., ICLR 2018]
14. Towards Accurate Binary Convolutional Neural Network [Lin et al., NeurIPS 2017]
15. Incremental Network Quantization: Towards Lossless CNNs with Low-precision Weights [Zhou et al., ICLR 2017]
16. HAQ: Hardware-Aware Automated Quantization with Mixed Precision [Wang et al., CVPR 2019]